

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I P. Gowtham Reddy(192525139) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Gowtham Reddy

Annexure A

Analytical Problem Solving

1. RSA Key Generation and Security Analysis

Given:

$$p = 17, q = 19$$

Compute n and $\phi(n)$

Choose $e = 5$ and calculate the private key d

Encrypt the message $M = 7$

Decrypt the ciphertext and verify correctness

Analyze the impact of choosing a small prime number on RSA security

ANSWER:

Definition

RSA (Rivest–Shamir–Adleman) is an asymmetric public-key cryptographic algorithm used for secure communication. It uses two keys:

- **Public Key** – Used for encryption.
- **Private Key** – Used for decryption.

RSA security is based on the difficulty of factoring the product of two large prime numbers.

Given

- Prime numbers: $p = 17, q = 19$
 - Public exponent: $e = 5$
 - Message: $M = 7$
-

Step 1: Compute n and $\phi(n)$

The RSA modulus is:

$$n = p \times q = 17 \times 19 = 323$$

Euler's Totient Function:

$$\phi(n) = (p - 1)(q - 1) = 16 \times 18 = 288$$

Answer:

- $n = 323$
 - $\phi(n) = 288$
-

Step 2: Calculate the Private Key d

Given:

$$e = 5$$

Find d such that:

$$d \times e \equiv 1 \pmod{288} \quad 5d \equiv 1 \pmod{288}$$

Using the Extended Euclidean Algorithm:

$$d = 173$$

Verification:

$$5 \times 173 = 865 \quad 865 \div 288 = 3 \text{ remainder } 1$$

Therefore,

$$865 \equiv 1 \pmod{288}$$

Private Key:

$$d = 173$$

Step 3: Encrypt the Message

Encryption formula:

$$C = M^e n$$

Substitute the values:

$$C = 7^5 3237^5 = 1680716807323 = 11$$

Ciphertext:

$$C = 11$$

Step 4: Decrypt the Ciphertext

Decryption formula:

$$M = C^d n M = 11^{173} 323$$

Using modular exponentiation,

$$11^{173} 323 = 7$$

Recovered Message:

$$M = 7$$

Since the decrypted message is equal to the original message,

Encryption and decryption are verified successfully.

Summary

Parameter	Value
p	17
q	19
$n = p \times q$	323
$\phi(n)$	288
Public exponent (e)	5
Private key (d)	173
Message (M)	7
Ciphertext (C)	11
Decrypted Message	7

Security Analysis: Impact of Choosing Small Prime Numbers

1. Small prime numbers produce a **small modulus (n)**, making RSA easy to break.
 2. Attackers can quickly factor **$323 = 17 \times 19$** , revealing the private key.
 3. The encryption becomes vulnerable to **brute-force and factorization attacks**.
 4. Real-world RSA uses very large primes (typically **1024-bit, 2048-bit, or 4096-bit**) to ensure security.
 5. Therefore, using small primes is suitable **only for educational demonstrations** and should **never be used in real-world cryptographic systems**.
-

Conclusion

RSA key generation involves selecting two prime numbers, computing **n** and **$\phi(n)$** , choosing a public exponent **e**, and calculating the private key **d**. For the given values, the public key is **(5, 323)** and the private key is **(173, 323)**. The message **7** is encrypted to **11** and successfully

decrypted back to 7, proving the correctness of the RSA algorithm. However, small prime numbers make RSA insecure because they can be factored easily, so practical implementations always use very large prime numbers for strong security.

2. ElGamal Encryption Analysis

Given:

$p = 23$, $g = 5$, private key $x = 7$, random $k = 3$, message $M = 10$

Compute the public key

Perform encryption to generate ciphertext pair $(C1, C2)$

Decrypt the ciphertext to retrieve the original message

Analyze how randomness (k) improves security in ElGamal

ANSWER:

Definition

ElGamal Encryption is an asymmetric public-key cryptographic algorithm based on the Discrete Logarithm Problem (DLP). It uses a public key for encryption and a private key for decryption. Unlike RSA, ElGamal introduces a random number (k) during encryption, making the same message produce different ciphertexts each time.

Given

- Prime number: $p = 23$
 - Generator: $g = 5$
 - Private key: $x = 7$
 - Random number: $k = 3$
 - Message: $M = 10$
-

Step 1: Compute the Public Key

The public key is calculated as:

$$y = g^x p$$

Substitute the given values:

$$y = 5^7 23^7 = 781257812523 = 17$$

Therefore,

Public Key = (p, g, y)

$$(5 \ 17)$$

Step 2: Perform Encryption

Calculate C_1

Encryption formula:

$$C_1 = g^k p C_1 = 5^3 23125 23 = 10 C_1 = 10$$

Calculate Shared Secret

$$y^k p 17^3 2317^2 = 289 \equiv 13 \pmod{23} 17^3 = 17 \times 13 = 221 \equiv 14 \pmod{23}$$

Shared Secret:

$$14$$

Calculate C_2

Encryption formula:

$$C_2 = M \times y^k p C_2 = 10 \times 14 23140 23 = 2 C_2 = 2$$

Ciphertext

The encrypted ciphertext pair is:

$$(C_1, C_2) = (10, 2)$$

Step 3: Decrypt the Ciphertext

Compute Shared Secret

$$s = C_1^x p s = 10^7 23$$

Using modular exponentiation,

$$s = 14$$

Find the Modular Inverse of 14 (mod 23)

We need:

$$14^{-1} \text{ mod } 23$$

Since

$$14 \times 5 = 70 \equiv 1 \pmod{23}$$

The inverse is:

$$14^{-1} = 5$$

Recover the Original Message

Decryption formula:

$$M = C_2 \times s^{-1} \text{ mod } p$$
$$M = 2 \times 5 \text{ mod } 23 = 10$$

Recovered message:

$$M = 10$$

The decrypted message matches the original message.

Summary

Parameter	Value
Prime (p)	23
Generator (g)	5
Private Key (x)	7
Public Key (y)	17
Random Number (k)	3
Message (M)	10
C_1	10
C_2	2
Ciphertext	(10, 2)
Decrypted Message	10

Security Analysis: How Randomness (k) Improves Security

1. A new random value k is chosen for every encryption operation.
 2. The same plaintext encrypted multiple times produces different ciphertexts, preventing pattern analysis.
 3. Randomness protects ElGamal against chosen-plaintext and replay attacks.
 4. Even if an attacker knows the public key and ciphertext, recovering the message without k or the private key is computationally infeasible due to the Discrete Logarithm Problem.
 5. If the same random value k is reused, the encryption becomes vulnerable, so k must always be unique and unpredictable.
-

Conclusion

ElGamal encryption uses the public key (23, 5, 17) to encrypt the message 10, producing the ciphertext pair (10, 2). During decryption, the receiver uses the private key to recover the original message successfully. The use of a fresh random number k for every encryption makes ElGamal highly secure by ensuring that identical messages generate different ciphertexts, providing strong protection against cryptographic attacks.

3. RSA Digital Signature Verification

Given:

$p = 11$, $q = 17$, $e = 7$, message $M = 8$

Compute n and $\phi(n)$

Find private key d

Generate the digital signature for the message

Verify the signature using the public key

Analyze how digital signatures ensure non-repudiation

ANSWER:

Definition

An **RSA Digital Signature** is a cryptographic technique used to verify the **authenticity**, **integrity**, and **non-repudiation** of a message. The sender signs the message using the **private key**, and the receiver verifies the signature using the **public key**.

Given

- Prime numbers: $p = 11, q = 17$
 - Public exponent: $e = 7$
 - Message: $M = 8$
-

Step 1: Compute n and $\phi(n)$

RSA modulus:

$$n = p \times q = 11 \times 17 = 187$$

Euler's Totient Function:

$$\phi(n) = (p - 1)(q - 1) \phi(n) = 10 \times 16 = 160$$

Answer:

- $n = 187$
 - $\phi(n) = 160$
-

Step 2: Find the Private Key d

Given:

$$e = 7$$

Find d such that

$$d \times e \equiv 1 \pmod{160} \quad 7d \equiv 1 \pmod{160}$$

Using the Extended Euclidean Algorithm:

$$d = 23$$

Verification:

$$7 \times 23 = 161 \equiv 1 \pmod{160}$$

Therefore, the private key is:

$$d = 23$$

Step 3: Generate the Digital Signature

The sender signs the message using the private key.

Signature formula:

$$S = M^d \pmod{n} S = 8^{23} \pmod{187}$$

Using modular exponentiation,

$S = 57$

Thus, the **Digital Signature = 57**.

Step 4: Verify the Signature

The receiver verifies the signature using the public key.

Verification formula:

$M = S^e \bmod n = 57^7 \bmod 187$

Using modular exponentiation,

$57^7 \bmod 187 = 8$

Recovered message:

$M = 8$

Since the recovered message matches the original message,

The digital signature is valid.

[\text{Learn more}](#)

Summary

Parameter	Value
p	11
q	17
n	187
$\phi(n)$	160
Public Exponent (e)	7
Private Key (d)	23
Message (M)	8
Digital Signature (S)	57
Verified Message	8
Signature Status	Valid

Security Analysis: How Digital Signatures Ensure Non-Repudiation

1. The sender signs the message using their **private key**, which only they possess.
 2. Anyone can verify the signature using the sender's **public key**, proving the message origin.
 3. Any modification to the message causes signature verification to fail, ensuring **data integrity**.
 4. The sender cannot deny sending the signed message because only their private key could have created the signature.
 5. Digital signatures are widely used in **electronic documents, online banking, software distribution, and e-commerce** to provide authentication and non-repudiation.
-

Conclusion

RSA Digital Signature uses the private key (**d = 23**) to generate a signature for the message **8**, producing the signature **57**. The receiver verifies it using the public key (**e = 7, n = 187**) and successfully recovers the original message **8**, confirming that the signature is valid. RSA digital signatures provide **authentication, integrity, and non-repudiation**, making them essential for secure digital communication and electronic transactions.

4. Diffie-Hellman with Attack Resistance Evaluation

Given:

$$p = 29, g = 2$$

User A private key = 5

User B private key = 12

Compute public keys for both users

Compute the shared secret key

Analyze what happens if an attacker intercepts and replaces public keys

Suggest a cryptographic solution to ensure authenticity

ANSWER:

Definition

The **Diffie-Hellman Key Exchange** algorithm is a public-key cryptographic protocol that enables two users to establish a **shared secret key** over an insecure communication channel. The security of the algorithm is based on the **Discrete Logarithm Problem (DLP)**.

Given

- Prime number: **p = 29**
 - Generator: **g = 2**
 - User A Private Key: **a = 5**
 - User B Private Key: **b = 12**
-

Step 1: Compute Public Keys

User A Public Key

Formula:

$$A = g^a pA = 2^5 292^5 = 323229 = 3A = 3$$

User B Public Key

Formula:

$$B = g^b pB = 2^{12} 292^{12} = 4096409629 = 7B = 7$$

Step 2: Compute the Shared Secret Key

User A Computes

Formula:

$$K = B^a pK = 7^5 297^2 = 49 \equiv 20, 7^4 = 20^2 = 400 \equiv 237^5 = 23 \times 7 = 161 \equiv 16 \pmod{29} K = 16$$

User B Computes

Formula:

$$K = A^b pK = 3^{12} 293^2 = 9, 3^4 = 81 \equiv 23, 3^8 = 23^2 = 529 \equiv 73^{12} = 3^8 \times 3^4 = 7 \times 23 = 161 \equiv 16 \pmod{29}$$

Both users obtain the **same shared secret key**.

Shared Secret Key

$$K = 16$$

Summary

Parameter	Value
Prime Number (p)	29
Generator (g)	2
User A Private Key	5
User B Private Key	12
User A Public Key	3
User B Public Key	7
Shared Secret Key	16

Attack Resistance Analysis

If an attacker intercepts the communication and **replaces the public keys**, a **Man-in-the-Middle (MITM) attack** can occur.

1. The attacker intercepts User A's public key before it reaches User B.
 2. The attacker sends their own public key to User B while pretending to be User A.
 3. Similarly, the attacker replaces User B's public key before sending it to User A.
 4. User A and User B each establish separate secret keys with the attacker instead of with each other.
 5. The attacker can decrypt, modify, and re-encrypt all messages without either user realizing it.
-

Cryptographic Solution to Ensure Authenticity

To prevent public key replacement attacks:

1. Use **Digital Signatures** to authenticate exchanged public keys.
 2. Employ **Digital Certificates** issued by a trusted **Certificate Authority (CA)**.
 3. Use authenticated key exchange protocols such as **TLS (Transport Layer Security)**.
 4. Verify the identity of both communicating parties before accepting public keys.
 5. Use Public Key Infrastructure (PKI) to ensure that public keys belong to the intended users.
-

Conclusion

In the Diffie–Hellman key exchange, User A computes the public key **3**, User B computes the public key **7**, and both derive the same shared secret key **16** without transmitting it over the

network. However, the protocol alone does not provide authentication and is vulnerable to **Man-in-the-Middle attacks**. Using **digital signatures, certificates, and authenticated key exchange protocols** ensures authenticity and protects the communication from attackers.

5. Hybrid Encryption System Analysis

Given:

A system uses RSA for key exchange and AES for data encryption

RSA encryption time = 5 ms per key exchange

AES encryption time = 0.5 ms per data block

Total data blocks = 500

Calculate total time taken for encryption

Compare with using only RSA for encrypting all data

Analyze why hybrid encryption is more efficient

Justify its use in modern secure communication systems

ANSWER:

Definition

A **Hybrid Encryption System** combines **RSA** and **AES** to provide both security and efficiency. **RSA** is used to securely exchange the AES secret key, while **AES** is used to encrypt the actual data because it is much faster than RSA.

Given

- RSA encryption time = **5 ms** per key exchange
 - AES encryption time = **0.5 ms** per data block
 - Total data blocks = **500**
-

Step 1: Calculate RSA Key Exchange Time

RSA is used only once to exchange the AES key.

$$RSA\ Time = 5\ ms$$

Step 2: Calculate AES Data Encryption Time

AES encrypts all data blocks.

$$AES\ Time = 500 \times 0.5 = 250\ ms$$

Step 3: Calculate Total Encryption Time

$$Total\ Time = RSA\ Time + AES\ Time = 5 + 250\ Total\ Encryption\ Time = 255\ ms$$

Step 4: Compare with Using Only RSA

If RSA were used to encrypt all 500 data blocks:

$$RSA\ Time = 500 \times 5 = 2500\ ms\ RSA\ Only\ Encryption\ Time = 2500\ ms$$

Comparison

Method	Time Taken
Hybrid Encryption (RSA + AES)	255 ms
Only RSA	2500 ms

Time Saved

$$2500 - 255 = 2245\ ms\ Time\ Saved = 2245\ ms$$

Hybrid encryption is approximately:

$$\frac{2500}{255} \approx 9.8\ About\ 10\ times\ faster\ than\ using\ only\ RSA$$

Analysis: Why Hybrid Encryption is More Efficient

1. **RSA** is computationally expensive and is suitable only for encrypting small amounts of data such as secret keys.
 2. **AES** is a fast symmetric encryption algorithm designed to encrypt large amounts of data efficiently.
 3. RSA performs only a single key exchange, while AES encrypts all data blocks quickly.
 4. This combination reduces computation time and improves overall system performance.
 5. Hybrid encryption provides both strong security and high-speed data transmission.
-

Justification for Use in Modern Secure Communication Systems

1. Used in **HTTPS/TLS** to establish secure web connections.

2. Used in **VPNs** for secure communication over public networks.
3. Used in **email encryption** (PGP/S-MIME) for secure message exchange.
4. Used in **online banking and e-commerce** to protect financial transactions.
5. Provides the security of asymmetric cryptography and the speed of symmetric cryptography, making it ideal for real-time communication.

Conclusion

A hybrid encryption system uses **RSA** to securely exchange the encryption key and **AES** to encrypt the actual data. For the given values, the total encryption time is **255 ms**, whereas encrypting all data using only RSA would take **2500 ms**. Thus, hybrid encryption saves **2245 ms** and is nearly **10 times faster**. Due to its combination of **high security, fast performance, and scalability**, hybrid encryption is widely used in modern secure communication systems such as **HTTPS, VPNs, secure email, online banking, and cloud services**.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate	Selects correct	Inappropriate or partially	Unable to select

		method/form ula with strong justification	method with minor errors	correct method	method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganize d steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretatio n of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretati on with minor omissions	Limited or unclear interpretatio n	No interpretati on